When the Monitor Blinds Itself: Failure Detection in Multi-Agent LLM Systems Is Limited by Observation—and Coupled to Its Cost

ANSHUMAN MULLICK, Bilkent University, Turkey
ERAY TÜZÜN, Bilkent University, Turkey

A multi-agent LLM system runs many workers in parallel against a single plan. When that plan quietly breaks mid-run, the workers keep spending tokens on a dead plan until the very end. We built a monitor to catch it, wrote down the bar it had to clear in advance, and ran it to a verdict we were not free to renegotiate. It failed—three times—and the failure is the finding.

The finding has two halves. First, catching a broken plan is limited by what the monitor gets to *see*, not by how clever it is—and a monitor that interrupts too eagerly spends on false alarms the very budget a second look needs, so accuracy and restraint draw on one finite pot. Second, the only mechanism that restores that second look—re-reading the plan and writing fresh checks—is also the system's largest cost, so cutting the cost risks the detection it buys: **detection and cost are coupled.**

Our first version failed badly: 35% detection, a "judge" stage that rubber-stamped its own false alarms, and a cost worse than the plain baseline it was meant to save—which a *naive* baseline then beat outright, at zero false alarms. A trace-level diagnosis, reviewed by two outside red teams, found it had starved its own observation. The redesign restores observation directly and catches 24 of 31 injected faults at zero false alarms—but at a 55.5% clean overhead against a 12% cap, an **overall fail** on cost, because that overhead *is* the plan-reading step that does the detecting, and spreading it over more workers only widens the gap. A separately and later pre-registered study then put a second vendor's model (GPT-5.5) in the check-writer's seat—a cheaper, better writer—and the bar failed a third time: shortening the checklist to save money drops exactly the faults that need a specific probe and *destabilizes* detection. The coupling holds across both model families.

Additional Key Words and Phrases: multi-agent LLM systems, runtime verification, self-adaptive systems, failure detection, interrupt policy, pre-registration, agent orchestration

1 Introduction

Twelve seconds before the fault it existed to catch, our monitoring system destroyed its own coverage.

Here is what happened in one run. A check fired on a perfectly healthy file listing. A "judge" model approved the alarm as real, at 0.98 confidence, by hallucinating a misreading of the listing. The orchestrator paused its workers and replanned. The new plan got a fresh set of checks—and those checks covered none of the surfaces the old ones had watched. Twelve seconds later the real fault fired into a world no one was watching. The run finished, confident and wrong.

Every stage did its job. The system as a whole blinded itself.

This paper is about that mechanism. In a multi-agent system on a fixed budget, catching a broken plan is bounded by *observation*, not by the monitor's *intelligence*—and a monitor that is too eager starves its own observation. We establish this through three acts: a pre-registered experiment that killed most of our design, a careful diagnosis of why, and a redesign that puts observation back at the center.

The stakes. Multi-agent LLM systems are expensive. They use, on the order of 15× the tokens of single-agent chat, as one production report puts it [27]. When a plan breaks mid-run, plain batch

Authors' Contact Information: Anshuman Mullick, Bilkent University, Ankara, Turkey, anshumanmullick7@gmail.com; Eray Tüzün, Bilkent University, Ankara, Turkey, eraytuzun@cs.bilkent.edu.tr.

2026. ACM 2994-970X/2026/7-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

orchestration only finds out at the end—after every parallel worker has spent that multiplier on a dead plan. These failures are real and well-catalogued [14, 16]. And the *mechanism* to interrupt a run already exists in production frameworks [22]. What is missing is the *policy*: something that decides what to watch for, whether an anomaly actually breaks the plan, and when interrupting is worth it.

What we built. We proposed a layer called Sentinel Protocol. The idea is simple. Before the workers run, an LLM reads the plan once and writes out a set of small, concrete checks—we call them *tripwires*—that say what each step is trusting about the world. The workers then watch for those checks cheaply, by pattern matching, and a confirmed break interrupts the orchestrator so it can replan. We pay for expensive reasoning once, up front, and keep the running system cheap. (For software-engineering readers: it is a classic self-adaptive control loop [23], but its monitor is *generated from the plan at runtime* rather than written by hand.)

What we did to it. We then did something the field rarely does to its own proposals. We wrote down pass/fail lines *in advance*—with the consequence of each failure decided ahead of time—froze them, and ran the experiment to a verdict we could not move after seeing the data.

The verdict killed most of the design. Detection was 35%, against a 60% bar and a 40% floor below which the claim died outright. The "judge" stage approved all 34 alarms it ever saw, including all 18 false ones—a rubber stamp, not a filter. The cost was worse than the plain baseline it was built to save. One claim survived: where a fault keeps showing up on a surface someone watches, our checks catch it about 3× faster than a cost-matched periodic re-check (median 3 vs. 9 tool calls).

Why it failed. A failed verdict is only useful if you find out why. We replayed the fully recorded runs, sent the diagnosis to two outside red teams for blind review, and settled every testable claim with a deterministic replay that cost no LLM calls. The diagnosis: detection here is observation-bounded, and in our case the bound was self-inflicted. On identical seeds, ordinary baseline systems re-looked at every starved surface (12 of 12)—but ours did not. Its own false alarms had eaten the part of the run in which a second look would have happened. Under a fixed budget, accuracy and restraint are not separate dials.

The redesign. The fix follows from the diagnosis: stop relying on workers passively noticing things, and instead have the monitor *re-read the world itself* with cheap, deterministic checks. We validated this mechanism in offline replay before building it. Then we re-measured the new version against a fresh set of pre-registered bars, frozen before any new data existed, with two fault types whose exact settings were sealed away with someone outside the project—so the design could not have seen them.

The new verdict. The redesign passed detection and failed cost. It caught 24 of 31 injected faults, at *zero* false alarms on clean runs, recovering exactly the quiet, hard-to-see fault types the first version was blind to. It even transferred partway to the sealed fault types it had never seen: it caught the held-out RESOURCE_BUDGET type 3 of 5 times, where every passive baseline caught it 0 of 5—a *directional* edge at $n = 5$, not yet a significant one (§7). (The other held-out type, DEPENDENCY_VERSION, was missed by *every* system, ours included—the same observation bound met on the read side: a fault that fires before the monitor's first look cannot be caught by comparison, by any system (§7).) But its extra cost on clean runs was 55.5%, against a 12% cap—an **overall fail**, driven entirely by cost.

So the thesis sharpens. Restoring observation fixes detection and ends the self-harm. But the restored observation also has to be *cheap*, and ours is not.

This is not a system that failed at its only job. Where signals recur, it detects about 3× faster than the obvious alternative. It injures *zero* clean runs, where both its own predecessor and the naive baseline raise false alarms. And it carries part of its skill to fault types it was never shown. The cost failure is not the absence of value. It is that the value does not yet clear the cost of looking.

Section 8 shows that this cost failure and the detection success are two views of the same fact: you cannot monitor a plan without reading it, and reading the plan is the irreducible cost. Section 9 shows the same fact with a different vendor's model in the check-writer's seat, under a separate frozen bar committed later—a cheaper, better writer, and the coupling holds anyway, now because *coverage* of the plan is both what costs and what catches.

Contributions.

- **The finding** (§6, §8, §9): in budget-bounded multi-agent execution, detection is limited by observation, that limit can be self-inflicted, and—because the one thing that restores observation, reading the plan to write checks, is also the main cost—detection and cost are *coupled*. The negatives are not a string of missed targets; they converge on one fact. That fact reproduces with a second vendor's model in the check-writer's seat (§9), through a different mechanism—checklist coverage, not reasoning effort: the coupling is not one vendor's artifact.
- **A pre-registration method for agent-systems research** (§5, §7, §9) that actually bound us: frozen bars returned *fail three* times—twice on both versions of our own system, and a third time in a separately and later pre-registered live study of a different vendor's model (§9)—and the pre-decided consequences ran as written each time. We add binding outside red-team review, a data embargo, a log of every run, sealed held-out fault types, and a check-writer that is never told the fault list.
- **A diagnosis method** (§6): deterministic replay over fully recorded runs, used to settle outside criticism with evidence rather than argument, and to test the redesign before building it.
- **TripwireBench** (§4): a benchmark of injected faults that are *verified to actually break a plain baseline*, with clear recovery labels, sealed held-out types, and cost/speed/noise metrics.
- **The surviving result and the naive baseline** (§5, §7): a 3× speed edge where signals recur, and a no-frills baseline that caught 56% at zero false alarms and beat our full first version in every category—the honest floor any successor has to clear.

2 Background and Related Work

Self-adaptive systems. Software-engineering readers will recognize the shape: a monitor–analyze–plan–execute control loop, a staple of self-adaptive systems for two decades [23, 24], including recent work that puts LLMs inside the loop [25, 26]. We do not claim the loop. What is new is where its parts come from: the monitoring logic is *written from the plan, at runtime, by an LLM* rather than designed by hand. Our experiment then shows that the analysis step is the hard part—an uncalibrated filter does not just fail to help; it does damage (§5).

The nearest prior work, confronted. The closest ancestor is RVPLAN [6, 33], which turns a planner's stated preconditions into runtime monitors and replans when one breaks. "Watch the plan's assumptions, interrupt to replan" is therefore established. But RVPLAN lives in a setting where three things hold that do not hold for us:

- *The specification already exists.* RVPLAN translates preconditions that sit, machine-readable, in a planning file. Our checks have to be *invented* from a plan written in natural language. Coverage is therefore bounded by what the writer can infer (§6).
- *The world narrates itself.* RVPLAN reads a clean stream of labeled facts in its own vocabulary. We read raw, unlabeled tool output, often only once—and the gap between "watching" and "actually seeing" is our central result.
- *The stream is clean.* In a labeled stream a precondition simply holds or it does not; nothing can false-fire, so nothing needs filtering. In our setting the filtering layer is the whole game, and it is where our first version broke.

An established paradigm meets the LLM-agent setting and breaks in three measurable ways. This paper is about what it takes to survive there.

Other lines. Runtime verification and monitor synthesis [4, 5] give us the spec-to-monitor tradition; we trade full temporal logic for a small predicate language over tool calls, and our complaint is about *passivity*, not the trade (§3). LLM-agent guardrails [9–13] enforce externally written *safety* rules; our checks come from the plan, target plan validity and cost, and feed replanning. Failure taxonomies [14, 20] catalog things agents do wrong; we cover the complement—things the *world* does that the plan didn't expect—and we use them to write checks before the run, not to label failures after. Observability tools [15–18] explain failures after the fact, for humans; we act on the same signals in flight. LangGraph's interrupt [22] is mechanism without policy; schedulers [19] consume interrupts once someone decides to raise them. We sit in the gap: deciding *what to watch* and *whether to pull the trigger.*

3 The System Under Test

This section describes the system, but the system is our *instrument*, not our contribution. It is the thing we pre-registered, killed, diagnosed, and rebuilt in order to reach the finding. We describe the current (v2) version and note where the first version (v1) differed. Every change from v1 to v2 cites the trace evidence that forced it (§6), and v1's verdict is reported in full no matter how v2 turns out (§5, §7).

Three roles. The *orchestrator* owns the plan and is the only part allowed to replan; it acts only on confirmed breaks. The *sentinel* runs once, up front: it reads the plan and writes the checks. The *workers* carry those checks and watch for them cheaply while they work, raising a structured alarm on a match. The cycle is: plan, write checks, run, raise an alarm, confirm-and-interrupt, replan. Pay for the thinking once; keep the running system cheap and deterministic. The pilot added one rule the new version follows: the running system also has to *look*—not just wait to be told.

Checks and probes. A check is concrete. It names a specific thing to watch (a status code, a field, a structure, a hash), a specific value, and what to do if it trips. Vague checks ("if the API changed") are not allowed. One rule we learned the hard way and now enforce: every constraint the check-writer must follow has to live in a place the model actually reads—a field description in the schema—because rules written in comments or prose simply do not reach it.

The pilot showed the checks were fine but *passive*—they only noticed what a worker happened to read. So the new version's key addition is the **active probe**. Each check can carry a small, deterministic re-read of the world (a HEAD request, a schema fingerprint, a status re-check) that runs on its own, with no LLM involved. Three things, each forced by what we measured (§6), shape how probes run:

- They run on a *separate channel* that does not disturb the world they measure—replay found three concrete ways a naive probe perturbs its own measurement.
- They fire *on events and risk*, not on a fixed clock—a naive fixed schedule once cost 2,322 probe calls on a 107-call run.
- A non-obvious alarm interrupts *only with a confirming probe*. The first version's "wait for a second signal" rule failed because correlated noise confirms itself—6 of 18 false alarms passed it. (A clear status-code error still interrupts on its own; no false alarm ever carried a status $\geq$ 400.)

Probes that compare a value need something clean to compare against. So at the very start of a run, before any worker acts, the system takes one clean reading of every surface it will watch, and later compares against that (deviation D30).

We removed the judge. The first version put an LLM "judge" between an alarm and an interrupt. Its record: it approved all 34 alarms it ever saw, ruled the only two genuine signals it down-ruled as noise, and once approved a hallucinated misreading at 0.98 confidence. We committed—in writing, *before* the diagnosis results were readable—to making the no-judge version the primary design (deviation D33 keeps a rebuilt judge only as an exploratory side arm). There is no after-the-fact selection here.

The check-writer is never told the fault list. The first version walked a list of eight named fault categories. The new version's check-writer gets *no list*. It reasons from two general ideas instead: what does this step trust about the world, and which of six general *shapes of change* might break it (a field vanished, a status moved, a structure changed, a value moved, an order scrambled, a relationship broke). Its only worked examples come from the five fault types we had already seen, chosen by a rule fixed before any prompt tuning (deviation D27). We attach category labels *after* detection. This is what makes the held-out test honest: the writer has to *transfer* general reasoning to a sealed fault type, not look it up (§7). The pilot's sharpest lesson is why this matters. Fault types differ enormously in how visible they are. The one type the first version caught well was the loud, repeating, status-coded one; the quiet, seen-once types were nearly invisible. The probe exists to make every type as visible as the loud one.

4 TripwireBench

To test a monitor, you need a world that breaks on cue.

TripwireBench is that world: a fully mocked, deterministic set of services (auth, inventory, pricing, shipping, a document store, a code repo), with seed-derived data and no clock values anywhere. Faults are exact state changes that fire at exact tool-call counts, so any run can be replayed byte-for-byte from its setup. Because a fault fires at the same *point in the work* across systems and seeds, batch and treatment runs are directly comparable. Four task types (API aggregation, repo migration, research synthesis, document pipeline) run on a real agent CLI; one piece of middleware counts calls, fires faults, checks tripwires, and writes a full trace.

Every fault has to actually hurt. No fault enters the experiment until it provably breaks the plain batch baseline: injected runs must fail while clean runs pass, under a rule fixed in advance. The pilot qualified all 9 fault–task pairs. Two more fault types were authored *after* the pilot and held out: a metered quota that only complains after the budget is already spent (RESOURCE_BUDGET), and a silent version bump that quietly truncates a default page of results (DEPENDENCY_VERSION). Their exact parameters were sealed with someone outside the project, so part of the scoring could not be over-fit. Both cleared the same bar—the quota fault on the first try, the version fault only on its third revision, with the two failed designs reported as findings rather than hidden. The qualification record is itself instructive: a competent batch planner kept *defeating* faults—by paginating defensively before it saw anything, or by reading collections once at the start so a later change landed after the only look—and *strengthening* the baseline un-qualified two faults that had qualified earlier, so wounds must be re-checked whenever the baseline improves (deviation D23).

The systems we compare. S1, plain batch (dispatch, wait, aggregate, one redo). S2, naive interrupt (workers raise any anomaly straight to the orchestrator—no check-writer, no judge). S3, a cost-matched periodic re-check, tuned so its overhead equals our monitor's—the strongest fair version of the obvious alternative. S4, our monitor without the judge. S5, our full first version. The confirmatory study runs four arms—S1, S2, S3, and the redesigned monitor—setting the unbuilt rebuilt-judge arm aside (deviation D33).

What we measure. Wasted work (tokens and dollars spent after a fault that don't reach the final answer); detection speed (time to detection, TTD: calls from fault to first pause); detection rate; false-alarm rate (false-interrupt rate, FIR: with no fault and no alarm scored as zero); extra cost on clean runs; and plain pass/fail task success—no LLM grading anywhere. Pause attribution follows a written rule, with a hand-audited sample (13 of 13 agreed).

Integrity machinery. The benchmark's credibility tooling is part of the artifact: a frozen pre-registration with cryptographic fingerprints, a numbered log of every departure, fault triggers that only the test harness can reach, a log of every run touching any treatment component, a data embargo before the bars were frozen, and sealed held-out parameters verifiable against a public hash.

5 The Pre-Registered Pilot and Its Verdict

Setup. 13 task variants (9 qualified injected pairs + 4 clean) × 5 systems × 3 seeds = 195 runs, all banked, with no malformed traces. Tooling was pinned, model assignments fixed per role, sessions isolated. Faults fire at the halfway point of each task's typical length, identical across systems and seeds. Before the main experiment, two independent outside models scored the written checks against a four-part rubric (95.1% agreement) and they passed at 89% coverage against an 80% bar. The four pass/fail bars—the kill gates, KG1–KG4—were frozen before the first run, each with its consequence decided in advance. The decision window after the verdict was itself pre-set: 48 hours, "do not negotiate with the data".

The verdict (computed once, shown verbatim):

```
KG1 recall: 35% (>=60%) | categories >=50%: 1/5 (>=4)            -> FAIL
KG2 FIR: S5=1.0 S2=0.0 S4=1.0 (S5<=0.5*S2 and S5<=0.7*S4)        -> FAIL
KG3 cost: S5 med $1.178952 vs S1 med $0.340831; success S5=4%
     S1=22%; clean overhead OVER (<=12%)                           -> FAIL
KG4 vs heartbeat: wasted med S5=15014.0 S3=7950 (>=20% better)
     | TTD med S5=3 S3=9 (>=2x)                                    -> PASS (TTD arm)
```

Plainly: the bars we froze returned *fail* on our own flagship, and the pre-decided consequences ran as written. Detection at 35% fell in the kill band (below 40%), so that claim died. The false-alarm gate failed, so we dropped the judge. The cost gate failed, which triggered a break-even fit that later killed the cost claim outright (§8). Only one thing survived: speed. Our checks caught the fault in 3 calls to the periodic re-check's 9. But the wasted-work side of that same gate failed (15,014 vs. 7,950 tokens): **speed did not turn into savings under the first version's machinery**.

Reading it carefully. "Detection" here means a confirmed, trace-attributable pause, defined before the bars computed. A more generous reading (counting alarms that didn't pause) recovers only two runs—so the low number is real silence, not a strict definition. By category: the one passing type (PERMISSION_AUTH, 5/6) is exactly the loud, repeating, status-coded one; the quiet types scored 2/6, 1/6, 1/6, and 0/3. The predicted near-zeros showed up.

The naive baseline, stated plainly and prominently. S2—workers raise any anomaly straight to the orchestrator, no check-writer, no judge—caught **15 of 27 (56%) at a false-alarm rate of zero**, and beat or tied our full first version in *every* category. Workers given no filter escalate rarely, and mostly correctly, at this scale. The folk worry that naive escalation means alarm spam—which our false-alarm gate had baked in—was simply false here. Sophistication actively hurt. S2 is now a required head-to-head arm, under a rule we committed to in advance: if the simple thing wins, we say so in the results, not a footnote.

It hurt healthy runs, too. On clean runs with nothing wrong, the full first version succeeded 3 of 12 times against batch's 9 of 12. Seven of the nine clean failures were self-inflicted: five runs ground to death adjudicating noise, and two replanned so early that there was nothing to carry forward. All 34 interrupts had been judge-approved; 18 matched no fault at all, and 3 fired on entirely clean runs. An uncalibrated monitor did not just fail to help. It did harm.

6 Why It Failed

Where the misses happened. We classified all 27 injected treatment runs along the chain *write check* → *fault fires* → *alarm* → *judge* → *pause* → *attribute*. Of 18 misses: 1 had no check written; **12 had a check that simply never fired—the dominant case**; 2 fired but never raised an alarm; 2 were ruled noise; 1 fault never fired. So the problem was not blindness at write time—coverage was near-complete once a measurement artifact was fixed. It was *silence at look time*.

We did not grade our own homework. We sent the diagnosis and the redesign to two outside red teams for blind review, then settled every testable claim with a deterministic replay over the recorded runs—with our commitments fixed before any replay result was readable, and at zero LLM cost. All 27 injected worlds replay identically, which is what makes this possible. The reviewers split roughly evenly on the eighteen claims they raised, which is exactly why we *adjudicated* their review with evidence instead of deferring to it. The decisive checks: the story that evidence reached the system and was lost is refuted (9 of 12 starved runs had *no* post-fault reading at all—nothing to lose); the story that false alarms replanned the surface away is partial (that exact signature appears 0 of 12 times, though 7 of 12 died of noise); and the story that runs simply died before reaching the fault is refuted (0 of 17).

The load-bearing result. Our first diagnosis blamed the world: some surfaces are only ever read once, so nobody could re-look. The same-seed analysis corrected that, and it is the strongest evidence for the "not about intelligence" half of the finding.

On identical seeds, the ordinary baseline systems re-looked at every starved surface—12 of 12—and our "smarter" monitor did not.

The dumb baselines saw what the smart monitor missed. What starved it was its own noise: every false alarm spends turns and budget that detection itself needed. One run logged 172 alarms on healthy surfaces before grinding to death. One mechanism, four symptoms: noise starved detection, the over-trusting judge made the noise, and the grinding wrecked clean runs and cost. The opening story (§1) is this in a single run.

Errata, kept visible. Two early claims were corrected by the replay and stay in the record with their corrections: the "read-once surface" story was wrong for its own example (the surface was read *after* the fault, so the original telling was backwards), and "one matcher defect" became a *class* of dead checks—84 checks that could never match any real path under the live dialect, fixed with a compile-time liveness test (deviation D24).

We tested the fix before building it. We mechanically built probes for all nine pilot faults and replayed them against the broken worlds. Probe-first confirmation blocked **all 18 false alarms**, and projected detection *rose* to 21 of 27, because probes recover what alarms lose. We are careful here: **every number in this paragraph is a ceiling under stated assumptions**—hand-built probes the real check-writer had not yet produced, fixed run paths, the same nine known faults. It is a feasibility test of the mechanism, and we quote it nowhere as a result.

7 The Redesign, Measured

The confirmatory study is the rerun path we wrote down *before* the pilot verdict existed: report the first version as it stands, diagnose from traces, change the design with cited evidence, and re-measure against fresh bars.

The integrity rules are in ink. Every shared threshold is inherited from the *pre-verdict* values—60% detection, the same category and floor rules, the 2× speed bar—so no bar was set by anything the diagnosis told us. New rules add a hard "probes must be valid" gate, a recovery-quality clause (detect-only, detect-and-recover, and detect-and-justified-stop are counted separately, so stopping cannot be dressed up as success), and *absolute* noise caps in place of the broken ratio gate. Fault settings were redrawn from fresh seeds the designers never saw. The two sealed fault types sit inside the scoring. A data embargo and a full run log stood throughout. And because the check-writer never sees the fault list, the sealed types are unseen even in the vocabulary it reasons with.

Result. The experiment ran to completion—**172 of 172 runs banked**, across four systems (S1, S2, S3, and the redesign)—and the bars computed once.

```
1bKG1 detection: recall 10/15 = 66.7% (>=60%, floor clear);
     5/5 seen categories >=50% (API 6/6, SCHEMA_DRIFT 3/3, AUTH 6/6,
     TOOL_CONTRACT 3/3, RETRIEVAL_INTEGRITY 3/3); probe-validity 10/10;
     recovery-quality 17/24 = 70.8%             -> detection sub-terms PASS
1bKG2 noise/self-harm: clean median FIR 0, P95 0, max 0; 0 grinds;
     pre-detection median FIR 0; clean success 8/12 = 66.7%        -> PASS
1bKG3 cost: V2 clean median $0.3642 vs batch $0.2342;
     clean overhead 55.5% (cap <=12%)                             -> FAIL
1bKG4 vs heartbeat: V2 wasted 7008 vs S3 6404 = 1.09x;
     waste-parity not met (S3 detects 0/31)                       -> FAIL
OVERALL                                                           -> FAIL
```

The verdict is **fail, driven by cost**—strong detection that misses the cost bar.

One reporting note we keep visible. An admissibility check (a byte-identity replay) came back 123 of 124 identical; the single difference is a harmless ordering quirk on the non-detecting periodic-re-check arm, characterized and logged as deviation D34. The gate *code* folds that check into the detection gate, so the combined gate prints `FAIL` even though all four detection sub-terms pass. We report it split out—detection passes; the admissibility check is fine bar one benign diff—and note the overall fail is unaffected, because it rests on cost.

The 12% cost cap was a judgment call, and it is not load-bearing. We fixed the clean-overhead cap at 12% in the pre-registration, before any data existed. It was a pre-committed judgment, not a quantity derived from a model. **[AUTHOR-INPUT: confirm whether the frozen pre-registration records any informal rationale for the 12% figure; if it records none, state that none existed.]** Nothing downstream turns on the exact number. The verdict is invariant to the cap's value: the redesign fails any cap below 55.5%, and the second-family floor of +17% (§9) fails any cap below 17%—every bar a deployer could plausibly have set returns the same verdict. Post-hoc, the cap is if anything *generous* to the monitor: located on the fitted break-even curve (Appendix B), a 12% clean overhead pays for itself only near a 41% fault rate even under an accounting that ignores replan cost, and at plausible fault rates (5–25%) the overhead that would break even is 1.5–7.3% (all post-hoc).

Why the redesign wastes more than the do-nothing heartbeat (a post-hoc autopsy). The redesign's median wasted work (7,008 tokens) exceeds the cost-matched heartbeat's (6,404) even though the heartbeat detects nothing. An additive decomposition of the mean gap (807 tokens) puts the entire excess in one bucket: re-dispatch rework. When the redesign detects a fault it replans and re-dispatches workers, discarding the partial output of the workers left on the doomed plan; that discarded rework averages about 1,055 tokens per non-clean run, appears in 8 of 31 cells, and the heartbeat—detecting nothing, replanning nothing—discards none of it. It is not sunk pre-fault cost: the redesign actually books *less* of that than the heartbeat (mean 2,278 vs. 2,806 tokens), because it stops early and truncates the exposure window the heartbeat runs to completion. So the 1.09× parity is the price of corrective action: the redesign pays to detect, stop, and re-plan around the fault, and the metric counts that recovery machinery as waste, while the heartbeat's cheaper run simply finishes on the broken plan and delivers the wrong answer. Excluding the re-dispatch rework, the redesign's median waste falls to 6,117 tokens, below the heartbeat's 6,404. The frozen 1.09× verdict line stands; the autopsy shows the gap is recovery cost, not the monitor wasting more.

Detection, in plain numbers. The redesign caught **24 of 31** injected runs (Wilson 95% lower bound 60.2%). The passive baselines caught **0 of 31** (both batch and the periodic re-check), and the naive baseline caught **12 of 31**; the redesign's edge over that naive baseline is significant (one-sided Fisher exact $p = 0.002$). On the five seen fault types it was perfect, and it finally caught the quiet, seen-once types the first version could not (§5). Two numbers describe the same runs from different angles: 24 of 31 *detected* (77%), and 10 of 15 (66.7%) on the gated subset—the subset restricted to cells that admit a recovery path, a rule frozen in the pre-registration **[AUTHOR-INPUT: confirm the gated-subset rule and its freeze provenance]**. They differ by *definition*, not contradiction; and the choice of subset is not outcome-determining, because 24 of 31 (77%) also clears the 60% detection bar **[AUTHOR-INPUT: verify the author endorses this arithmetic claim]**. The 24 detections split into 3 detect-and-recover, 14 detect-and-justified-stop, and 7 detect-only, with 17 of 24 (70.8%) in the two passing buckets. The five seen categories are each perfect, but three sit at n=3, where the Wilson 95% lower bound is only about 44% and one run swings the rate 33 points—always reported with the bound.

It transferred, directionally, to a fault type it had never seen. The redesign caught the sealed RESOURCE_BUDGET type **3 of 5** times (Wilson 95% lower bound 23%), where every passive baseline caught it **0 of 5**. With $n = 5$ the one-sided Fisher exact test does not reach significance ($p = 0.08$), so we read this as a *directional* transfer of general reasoning to a fault type held out of its training entirely—not an established effect.

The second sealed type, DEPENDENCY_VERSION (a silent version bump that truncates a default page), was missed by *every* system, ours included (**0 of 5**). Read-only trace checking over all 20 of those runs explains why. The fault fires on the very first tool call—*before* the redesign takes its clean opening reading—so even our reference reading is already truncated, and the later re-reads match it. And no surface anywhere carries a version field, so the passive and naive baselines have no error to flag either. Because *every* system missed it—including the anomaly-raising naive one—this is the observation bound of §6 meeting the design on the read side, not a wart specific to our monitor. Stated as a scope condition over *all* the detections: a reference-comparison monitor catches a fault only if it lands *after* the monitor's opening reading and *before* its last look. The benchmark's main faults fire at the halfway point—inside that window by construction—which is why they are catchable at all; DEPENDENCY_VERSION fires on the first tool call, before the window opens, which is why it walls every comparison-based system, ours included.

The first version's pathology, reversed. The first version's signature—a rubber-stamped flood of false alarms that starved detection and hurt clean runs—is gone. Clean-run false-alarm rate is 0 (median, 95th percentile, and max), with zero grind-deaths. Its clean-run success (8 of 12) trails batch's (9 of 12) by a single run, and all four of its clean failures occurred at *zero* interrupts—worker-deliverable shortfalls (missing citations, unset fields, validation not run) that the no-monitor baseline shows too, so the monitor neither caused nor could have prevented them. And the win holds under real pressure: the clean set includes the legitimate config-migration write that used to trip a false alarm, and the redesign ran it at zero interrupts—while the naive baseline raised false alarms on four clean runs. The redesign bought back detection *and* dropped noise to the naive baseline's zero floor. Against the naive baseline's 12 of 31, sophistication helped this time. But the help did not survive the cost gate—which is the subject of the next section.

8 The Coupling: One Result, Not Three Failures

Start with the prediction economics makes. A cost paid *once per run*—read the plan, write the checks—should amortize across a large fan-out and all but vanish per worker: one fixed charge spread over many parallel workers. That is the natural reason to expect plan-compiled monitoring to pay for itself at scale. The measurement returned the opposite direction, and this section is why.

The cost failures of the first and second versions are not two missed targets. They are one fact, seen from a few sides: **detection and the cost of restoring observation are tied together**, because the one thing that restores observation is also the main cost.

The cost is the capability. We broke the 55.5% clean overhead down call by call. Almost all of it is one new component: the once-per-run LLM step that reads the plan and writes the checks (median $0.1376 on clean runs, against a $0.130 gap over batch), present in all 12 clean runs. The whole detection machinery—the probes, the re-readings, the confirmation logic—costs essentially nothing measured. So the overhead lives in the one step we cannot remove without removing the capability: that step *is* the reasoning that produces both the detection and the transfer to unseen fault types (§7). (§9 sharpens this: on a second model family the load-bearing content of that step turns out to be its *coverage* of the plan's assumptions, not its reasoning effort.) Making it cheaper—caching it, shrinking it, skipping it—is not a free optimization. It risks the detection it pays for. That mutual dependence is the coupling. A no-check-writer variant makes the point from the other side: drop that step and the system is nearly cost-neutral with batch on clean runs—so the step really is the cost center, though that variant's detection is only established on its narrow home case, so we report it as cost isolation, not a free lunch.

The obvious fix doesn't open. A fixed per-run cost should shrink, per worker, as you add more workers. So the natural escape is to spread that one-time check-writing cost across a bigger fan-out. The break-even model even predicts a crossover for the redesign—but that is a model prediction, not a measurement (Appendix B).

We built a width-scaling benchmark and ran a sizing pilot to test it. The pilot's point estimate—kept out of the formal sizing by design—is that the redesign is **costlier than batch at every width and fault rate we tried, and the gap *grows* as we add workers**, because the check-writing scales with the planning the system does, not with the parallel work it could dilute against.

We treat this as a *closing negative*, not a confirmation: the escape shows no sign of opening, and nailing it down at the pre-registered precision would cost about ten times the study budget ($\approx$ $4,430 against a $450 budget) for a number already pointing the wrong way—so a full fan-out study is not warranted. The price tag is the side note. The *direction* is the finding.

That closes the width escape; a depth escape we do not close. The sizing pilot varies *width*, the number of parallel workers. A second escape is *depth*: on longer tasks, execution tokens might grow faster than the plan, so the fixed check-writing cost could amortize against a longer run rather than a wider one. Our benchmark does not vary task depth enough to settle this—across the four task types execution size is nearly constant while plan size varies threefold and largely independently of it (Pearson 0.30, post-hoc)—so we do not claim the depth escape is closed. What the data does show is the scope condition: the plan is a non-trivial fraction of execution in every task type (0.32 to 0.86 of worker tokens). We therefore scope the coupling to runs whose plan is a non-trivial fraction of execution—which is precisely the multi-agent fan-out regime this paper targets.

And it fails even when looking is free. Our benchmark is a deterministic mock, where re-reading the world costs nothing. That is the *most generous* setting the system could ask for—it charges nothing for the very looking the system exists to do. And it *still* misses the budget, on the one-time check-writing step alone. So the measured 55.5% is a *floor*: in a real deployment, probes hit real endpoints at real cost, and the volume that reads as free here comes back as real overhead. A system that cannot pay for itself where looking is free will not pay for itself where looking costs money.

Putting it together. Restoring observation fixes detection (§7) and ends the false-alarm self-harm (§7). But the only place to cut cost is the capability-critical check-writing step, the obvious amortization does not open, and the failure holds even with looking priced at zero. So in budget-bounded multi-agent execution, **detection and cost are coupled, and this kind of monitoring does not yet pay for itself.**

The coupling has one root, on both model families. You cannot monitor a plan without reading the plan, and reading the plan is the irreducible cost. On the incumbent writer that cost shows up as a reasoning step; on GPT-5.5 (§9) reasoning is cheap and detection-free, and the floor is instead the writer's input tokens—the plan itself—spent through checklist coverage. The claim is not that thinking is expensive. It is that *coverage of the plan's assumptions has a floor cost proportional to the plan*, and coverage is what does the catching.

The verdict is conditional on how value is priced. The coupling is a claim about the *cost* side and holds under any value model. "Does not yet pay for itself" is a claim relative to a *token-denominated* value model—the one every bar in this paper uses. But the 14 detect-and-justified-stop runs each prevented a confidently-wrong deliverable—the exact outcome of this paper's opening run—and every bar here prices that prevented error at zero. Under a correctness-denominated value model the economics are open and unquantified here; pricing that term is the next experiment. The bars measure the token refund, and no one buys insurance for the refund. That next problem—and it is a fresh experiment, not a patch—is what the coupling leaves open.

9 The Coupling Holds on a Second Model Family (Live)

Everything above is one model family: a Claude check-writer, Claude and Haiku workers. A fair objection is that the coupling is that family's artifact. So we tested it on a second one.

This study was pre-registered separately, and later. After the original pre-registration and both its verdicts, on 2026-07-01, we froze a new, standalone pre-registration for this study alone—its own 12% bar, its own arms, its own pre-decided consequences, its own hard spend cap, all committed before any of its data existed. It is not part of the original plan and we do not present it as one: it is an additional, later-dated extension that reruns the paper's method on a different vendor's model. Folding it into the original frozen pre-registration would break the exact discipline this paper is about, so we keep the two apart and dated.

We swap GPT-5.5 (pinned `gpt-5.5-2026-04-23`; $5.00 / $0.50 / $30.00 per million input / cached-input / output tokens, verified 2026-07-01 [37]) into the check-writing step through the same frozen seam the redesign already exposes. Everything downstream—grounding, probes, confirmation, scoring—is byte-identical; only the writer changes.

Offline, the escape looked open. GPT-5.5 is a cheaper and cleaner writer than the incumbent. It writes valid checklists at $0.115 per call against the Claude writer's $0.1376 (§8), and it emits the value-comparing check that the cheap alternative never does (median 8 such checks per checklist, against 0 for a budget model). We first scored its checklists in the zero-LLM deterministic replay of §6. There, detection is *flat* as we turn its reasoning effort down—`none`, `low`, and `medium` all catch the same 16 of 18 seen faults—because the reasoning tokens are not what does the detecting. At `effort=none` it costs $0.064 a call, well under the incumbent, detection intact. On the reproducible replay, the coupling looked beaten.

Live, it was not. The replay prices looking at zero and holds the world byte-identical; it cannot see what a shorter checklist costs in a real run. So we ran the priced matrix live—real workers, real timing—and measured the overhead the bar is written on. The cost lever that could actually clear 12% is not reasoning effort (which is detection-free) but *checklist coverage*: a shorter checklist is a cheaper writer call. We capped it with a single general instruction, committed in the pre-registration and blind to the fault list—keep the N most load-bearing assumptions, judged only by how badly

Table 1. Live clean-run overhead by checklist-coverage cap (GPT-5.5 writer, reasoning fixed at `low`). Batch (S1) clean median \$0.2219 is the denominator, measured live. Single seed × 4 tasks (departure D-V3-1); the writer/batch-worker column is the robust reading (see text).

coverage	assumptions (median)	writer \$ (median)	overhead (frozen)	overhead (writer/batch)
full	40	0.093	+33.9%	+42.0%
cap-12	12	0.049	+20.6%	+21.9%
cap-6	6	0.038	+22.4%	+17.1%

Table 2. Live detection across three seeds (injected cells): the number of seeds, out of three, in which the fault's interrupt fired during full worker execution, by checklist-coverage cap. Five catchable faults plus the DEPENDENCY_VERSION wall. The bottom rows give catchable-fired-of-five per seed, then the median and range.

injected fault	full	cap-12	cap-6
removed endpoint (API_SURFACE)	3/3	3/3	**0/3**
expired token (PERMISSION_AUTH)	3/3	3/3	3/3
skipped gate (TOOL_CONTRACT)	3/3	3/3	2/3
swapped passage (RETRIEVAL_INTEGRITY)	**2/3**	**1/3**	**1/3**
RESOURCE_BUDGET (quota cliff)	3/3	3/3	2/3
DEPENDENCY_VERSION (silent bump)	0/3	0/3	0/3
catchable-of-5, per seed	5, 4, 5	4, 5, 4	3, 1, 4
median (range)	5 (4–5)	4 (4–5)	3 (1–4)

the plan breaks if each is false, never by what kind of fault it represents—at three frozen settings (Table 1).

No setting clears 12%. The raw (monitored − batch)/batch median is noisy at this study's single seed—worker cost varies run to run and swamps the writer saving, so cap-6 even reads worse than cap-12 on that column—so we read the overhead the robust way, as the writer call against the batch worker cost it adds to: +42%, then +22%, then +17%, every setting above the bar. The floor is the writer's own input tokens; even a six-assumption checklist costs \$0.038.

And the cap that lowers cost costs detection—and, extended to three seeds on the injected cells, *destabilizes* it (Table 2).

The direction is robust; the staircase is not. The load-bearing claim holds on every seed: the short checklist catches less than the full one—5, 4, and 5 of the five catchable faults at full, against 3, 1, and 4 at cap-6, so full beats cap-6 on all three draws. At the median the slope is a clean 5, 4, 3. But the per-rung counts are seed-variable, and prominently so: cap-6 ranges from 1 to 4 of 5, and one seed is non-monotone (4 at full, 5 at cap-12, 1 at cap-6). The tidy staircase is an artifact of a single draw; the *descent* is not.

Cheapening the monitor does not merely lower detection—it destabilizes it. At full coverage detection is fairly reliable (4–5 of 5); by cap-6 it is timing-dependent (1–4 of 5). The cap does not walk a steady dial down; it turns a dependable monitor into a gamble on whether six probes happen to land where and when the fault fires. The instability is itself a cost of cheapening, and it is a finding, not noise.

And the fragility sits where the theory puts it. The value-shaped fault (a swapped passage) is the one needing a probe on a *specific* surface at the *right* time—the closest fault to the observation bound of §6. It is marginal even at full coverage (2 of 3 seeds), and 1 of 3 at both caps: the hardest-to-see fault is the first to wobble and the first to fall. That is the coupling's own logic surfacing as variance, not a wart in it. By contrast the removed endpoint is robust until it is not (3 of 3 at full and cap-12, then 0 of 3 at cap-6, where six assumptions no longer reach the pricing surface at all); the auth fault, which 401s every surface at once, fires 3 of 3 everywhere; and the version bump stays the §7 wall (0 of 3 at every setting, excluded from the slope). The one setting that catches all five on a good draw is the full checklist—which has the *worst* overhead. Cheapening the writer loses faults, makes the rest unreliable, and *still* misses the bar.

These three draws are not a controlled seed replication, and we do not claim one. The dominant source of the run-to-run variation is the *live* non-determinism of the unseeded orchestrator, workers, and writer; the world seed adds fixture and token-stream variation on top. That is the right variation for this question—it is exactly what made the live numbers timing-variant—so the draws test robustness to deployment reality rather than to a re-seeded fixture. It is not a claim of three independent controlled trials.

Verdict, against the frozen V3 bar: the coupling holds (live). Both committed failure conditions are met—detection erodes as cost drops, and no setting clears 12%. This is the paper's thesis on a second model family, and it arrives through a *new mechanism.* On our own system the cost center was the one reasoning step (§8); on GPT-5.5 the reasoning is cheap and detection-free, and the cost lever is instead the checklist's *length*—which is its coverage. Different mechanism, same coupling: the thing you would shorten to save money is the thing that does the catching. The priced matrix and the multi-seed extension together cost \$6.54 in metered spend, under the pre-registered cap.

What this is and is not. It is a fail on the bar, reported as one, not an escape. GPT-5.5 is genuinely the better writer; it does not clear the cost. Four cautions survive verbatim. (1) The clean cost cells are a single seed (departure D-V3-1); the injected detection cells were extended to three seeds. The robust signals are the writer-cost overhead (clean) and the detection *direction* (full beats cap-6 on all three seeds); the per-rung counts are seed-variable, and we report that instability above as a finding rather than bury it as noise. (2) Prompt-caching did not engage this run; a warm cache could push the six-assumption cost floor toward 6%, but that was not achieved here and is future work, not a result—and even a cost pass would not matter, because at cap-6 detection already loses faults and is unstable (1–4 of 5), so the detection-axis coupling stands regardless. (3) The offline replay and the live run are different measurements and we do not blend them; the offline optimism was contradicted live at the load-bearing point, and that contradiction is part of the finding, not an embarrassment. (4) The DEPENDENCY_VERSION wall of §7 stands for GPT-5.5 too— missed at every setting, every seed, constant across arms, and so excluded from the coverage slope; and RESOURCE_BUDGET, which reads 8 of 8 in the coverage-bound offline replay, fires live at full and cap-12 but only 2 of 3 seeds at cap-6— seed-fragile once the checklist is shortest. Each is labeled as what it is.

10 Threats to Validity

These are findings, not laws. One harness, one mocked world, three seeds, bars written on medians. The workers are one model family throughout; the *check-writer*—the coupling's cost center—we now test on two (§9), where the coupling reproduces through a different mechanism. Plan shape varies run to run and we do not control it. Our injected faults approximate, but do not sample, the real distribution; qualification guarantees each one breaks the baseline, not that it is representative. We state the observation and cost results as regularities of this setting, not proofs.

The free-looking mock cuts both ways. It makes the cost verdict *conservative*—the system fails the budget where looking is free (§8)—but it cannot settle real-deployment cost, where probe volume and latency return. The 55.5% is a floor, and the one cost lever is also the capability, so the cost fix carries detection risk. Both are named; neither is solved. The 12% bar itself was a disclosed pre-registered judgment, not a derived quantity, and the cost verdict does not turn on it: every cap a deployer could plausibly set returns the same fail (§7).

The mock generates no benign noise, and that bounds the other verdict. The same free-looking mock that makes the cost verdict conservative also emits *no* benign anomalies—no transient 500s, no rate-limit jitter, no flaky reads, no latency spikes. So the zero-false-alarm results—the redesign's clean false-alarm rate of 0 and the naive baseline's—are measured in the most benign environment possible and are *upper bounds* on deployment behavior, not deployment numbers. The cascade is honest: under real noise the §6 self-starvation mechanism could re-open—false alarms eating the budget a second look needs—so the confirmatory detection numbers inherit the same qualification as the false-alarm rate. And the probes are not side-effect-free in the field: here a probe costs nothing and perturbs nothing, but a probe against a *metered* surface spends the very quota it checks, so the RESOURCE_BUDGET probe is potentially self-defeating live. The mock is generous to the noise result exactly as it is harsh to the cost result, and this paper leans on both directions: we accept the conservative cost verdict, so we must flag the optimistic noise one. **[AUTHOR-INPUT: A7 (benign-noise smoke) is pre-registered but not yet run; if run, replace this concession with the measured bound and cite the A7 mini pre-registration.]**

The redesign learned from the pilot's traces. That is a real leak, and we mitigate it rather than hide it: fresh fault settings the designers never saw, sealed held-out types (drawn by an unseeded cryptographic process with a committed hash) inside the scoring, and a check-writer whose blindness to the fault list is verifiable by inspection. The held-out types are held out in their settings, not their *kind*—same hands, same ontology—and the baseline defeated two of three designs before one qualified, which shows authoring genuinely wounding faults is itself hard. A held-out-*ontology* arm and a natural-failure arm remain the main generalization tests for future work.

We built the benchmark, the system, and the bars. The counterweight is the apparatus: frozen pre-registration, pre-decided consequences, cryptographic fingerprints, binding outside review with trace-level rulings, a public hash, and negatives reported at full prominence—including a naive baseline beating our own design (§5). The strongest evidence it binds is behavioral: frozen bars returned a kill verdict three times, across two model families—twice on our own system and, under a separately and later frozen pre-registration, a third time on a different vendor's model (§9)—and the pre-decided consequences ran as written every time.

11 Conclusion

We pre-registered a system we believed in, and the bars we froze killed most of it. We diagnosed why from fully recorded runs, rebuilt the detector around the diagnosis—and the bars killed the rebuild too, on cost. Because each verdict came with its mechanism instead of a shrug, they compose into one finding.

In budget-bounded multi-agent LLM execution, catching a broken plan is limited by observation, not by the monitor's cleverness: the system that re-looks, detects, and the plain baselines that kept re-looking saw what the clever one starved itself out of seeing. And because the one thing that restores observation—re-reading the plan to write fresh checks—is also the main cost, detection and cost are coupled. Restoring observation fixes detection and ends the self-harm. It does not yet pay for itself, and the obvious way to make it pay does not open, even when looking is free.

And it is not one model's artifact. A separately and later pre-registered live study put a different vendor's model in the check-writer's seat—a cheaper, better writer—and the coupling held: the bar failed a third time, now through the checklist's length rather than its reasoning (§9).

That coupling is a claim about cost, and it holds however value is priced. What it leaves open is the other side of the ledger: every bar here is token-denominated, and each of the fourteen justified stops prevented a confidently-wrong deliverable—the outcome of the opening run—priced at zero throughout. Under a correctness-denominated value model the economics are open, and pricing that term is the next experiment.

That coupling, not any single missed bar, is the result. It is what a plan-driven monitor has to solve next.

Data Availability

A replication package—the benchmark world and harness, the frozen pre-registration with fingerprints, all 195 pilot traces and the deterministic replay battery, the 172-run confirmatory experiment and its report, the cost autopsy, the fan-out sizing pilot, the qualification and review records, the departure log, and the run log—is at [REPLICATION-PACKAGE-URL]. It also includes the second-family study of §9: its own, separately and later frozen pre-registration (dated 2026-07-01), the live priced matrix and per-run cost and detection records, and the offline replay scores. The held-out parameter values stay sealed with someone outside the project; their fingerprint is committed publicly so the file cannot be swapped.

A Integrity and Departure Log

Load-bearing departures, in brief; full records in the departure log (Data Availability). **D23** removed two conditionally-qualified runs from the experiment (215 → 172 with four arms) and named the *write-side single-look* residual—an agent's own write can overwrite a change it never saw. **D24** fixed the dead-check class (84 checks, instrument-level, with a regression test). **D25/D27** fix the held-out scoring outside the project and freeze the check-writer's example-selection rule before any tuning. **D28–D32** record the confirmation, scheduling, opening-reading, write-handling, and grounding rulings of the rebuild, each committed before its code and each preserving identical behavior when the flag is off. **D33** set the unbuilt rebuilt-judge arm aside. **D34** characterizes the lone replay difference (123/124, benign, on the non-detecting arm); no number or the overall fail changes.

The second-family study (§9) is pre-registered on its own, and later. Its frozen pre-registration is dated 2026-07-01—after the original and both its verdicts—with its own 12% bar, arms, coverage settings, verdict definitions, and a hard spend cap, all committed before its data existed and never folded into the original pre-registration. **D-V3-1** logs its execution departures: the clean cost cells ran at a single seed (four tasks) with a reduced injected set, executed in resumable foreground chunks after the background runner proved unreliable—execution-only changes that leave the frozen bar, lever, verdict meanings, and spend cap untouched. A subsequent robustness extension re-ran the *injected detection* cells at two further seeds (three in total), reported in §9; the direction of the detection slope is robust across all three while the per-rung counts are seed-variable. The study spent \$6.54 of metered budget across matrix and extension, under its \$12 cap; no cap was hit.

B Fan-out Break-Even Model (a prediction, not a result)

Cost-positivity holds when $C + J + p \cdot R < p \cdot (W_{\text{batch}} - W_{\text{sent}})$. Fitting the model to the first version's runs (C = \$0.322 check-writing, J = \$0.048 judge, R = \$0.257 per replan) gives a *negative* waste gap of −\$0.072 and **no crossover at any fan-out, with probability 1.00 over 1,000 resamples**. Re-fitting to the redesign gives a positive gap and an extrapolated crossover near 86, 40, and 25

Table 3. Post-hoc: maximum clean overhead at which redesign monitoring is cost-positive, by fault rate p, under the generous waste-recovery reading ($M < p \cdot \Delta W$; ΔW = \$0.068, baseline \$0.234). The pre-committed model including replan cost admits no positive break-even at the measured fan-out ($n_0 = 3$).

fault rate p	0.05	0.10	0.25	0.50
max clean overhead	1.5%	2.9%	7.3%	14.6%

workers at fault rates 0.10, 0.25, and 0.50. **These are model extrapolations—predictions to test, never measured results.** The sizing pilot (§8) tested the direction and closed it negatively.

Locating the 12% cap on this curve (post-hoc). Inverting the same model for the maximum clean overhead a deployer could tolerate—rather than for a crossover fan-out—does not admit a clean answer at the *measured* fan-out. At $n_0 = 3$ the redesign's per-replan cost (R = \$0.221) already exceeds its per-run waste gap ($W_{\text{batch}} - W_{\text{sent}}$ = \$0.068), so $C + p \cdot R < p \cdot \Delta W$ fails for every fault rate and every overhead including zero: a *free* monitor is cost-negative at $n_0 = 3$, turning positive only past a fan-out of about ten workers. The nearest meaningful locator drops the replan term—asking only whether the clean overhead is recovered by the expected saved waste, $M < p \cdot \Delta W$, which is generous to the monitor—and puts the 12% cap at a fault rate of $p^\star \approx 41\%$ (Table 3). Even under that generous reading 12% was, if anything, generous to the monitor, and the measured 55.5% is far outside break-even either way. All figures here are post-hoc.

References

[1] Execution monitoring survey (Dunlap et al.).
[2] Davis-Mendelow et al. Assumption-based planning. AAAI 2013.
[3] Bercher et al. Plan repair. ICAPS 2014.
[4] Bauer, Leucker, Schallhart. Runtime verification for LTL and TLTL. ACM TOSEM.
[5] Havelund, Rosu. Synthesizing monitors for safety properties. TACAS 2002.
[6] Ferrando, Cardoso. RVPLAN: Runtime Verification of Assumptions in Automated Planning. ICAART 2022, Vol. 2, pp. 67–77. DOI 10.5220/0010776500003116.
[7] Parallelized planning-acting. arXiv:2503.03505.
[8] Weak-to-strong monitoring. arXiv:2508.19461.
[9] AgentSpec. ICSE 2026. arXiv:2503.18666.
[10] Agent-C. arXiv:2512.23738.
[11] Pro2Guard. arXiv:2508.00500.
[12] ProbGuard. arXiv:2602.19844.
[13] LlamaFirewall. arXiv:2505.03574.
[14] MAST: multi-agent system failure taxonomy. arXiv:2503.13657.
[15] AgentOps. arXiv:2411.05285.
[16] Wasted-computation diagnosis in multi-agent systems.
[17] LumiMAS. AAMAS 2026. arXiv:2508.12412.
[18] DiLLS. CHI 2026. arXiv:2602.05446.
[19] Graph Harness. arXiv:2604.11378.
[20] SHIELDA. arXiv:2508.07935.
[21] Learning to Interrupt. arXiv:2604.06452.
[22] LangGraph interrupt documentation.
[23] Kephart, Chess. The vision of autonomic computing. IEEE Computer, 2003.
[24] Weyns. Introduction to self-adaptive systems. 2020.
[25] Nascimento et al. arXiv:2307.06187.
[26] ACM TAAS roadmap. DOI 10.1145/3686803.
[27] Anthropic Engineering. How we built our multi-agent research system.
[28] Mullick, Tüzün. [Prior work by the authors; citation to be completed at submission.]
[29] Mullick, Tüzün. [Prior work by the authors; citation to be completed at submission.]
[30] Zep temporal knowledge graphs. arXiv:2501.13956.

[31] GAIA. arXiv:2311.12983.
[32] tau-bench. arXiv:2406.12045.
[33] Ferrando, Cardoso. RVPLAN: a general purpose framework for replanning using runtime verification. VORTEX@ISSTA 2021, pp. 22–25. DOI 10.1145/3464974.3468447.
[34] Bozzano, Cimatti, Roveri, Tchaltsev. A comprehensive approach to on-board autonomy verification and validation. IJCAI 2011, pp. 2398–2403.
[35] Bensalem, Havelund, Orlandini. Verification and validation meet planning and scheduling. STTT 16(1):1–12, 2014.
[36] Ferrando, Cardoso. Runtime monitoring of action specifications for replanning in classical planning. VORTEX 2025.
[37] OpenAI. GPT-5.5 model reference and API pricing. developers.openai.com/api/docs/models/gpt-5.5 and /api/docs/pricing (model `gpt-5.5-2026-04-23`; $5.00/$0.50/$30.00 per 1M input/cached/output tokens; verified 2026-07-01).